Skill-11

Name: k.vaishali

Rollno: 2520030117

Section: 8

Experiment No. 11 (Q11)

Title: Command History Management and Pipeline Structures in a Custom Shell

Aim:

To implement a command-history mechanism that stores commands, maintains a history buffer, handles capacity limits, retrieves entries, displays history, and validates consistency. To implement pipeline structures that create a pipeline, store commands, maintain execution order, connect processes, validate pipeline layout, and test different configurations.

Objective:

Part 1 – Command History

- To store commands entered by the user.
- To maintain a fixed-size history buffer.
- To handle capacity limits once the buffer is full.
- To retrieve individual entries from the history.
- To display the stored command history.
- To validate the consistency of the history buffer.

Part 2 – Pipeline Structures

- To create a data structure representing a command pipeline.
- To store each command that makes up a stage of the pipeline.
- To maintain the execution order of the pipeline stages.
- To connect processes so one stage's output feeds the next stage's input.
- To validate the pipeline layout before execution.
- To test different pipeline configurations, including invalid ones.

Theory:

1. Command History

A shell typically keeps a record of previously entered commands so the user can review or re-run them. This record is stored in a history buffer, usually with a fixed maximum capacity to bound memory usage.

2. Circular Buffer for History Storage

Because the number of commands a user enters is unbounded, the history buffer is implemented as a circular buffer of fixed size. Once the buffer is full, the oldest entry is overwritten by the newest command, so the buffer always holds the most recent N commands.

3. Handling Capacity Limits

When the buffer reaches its maximum capacity, new commands overwrite the oldest stored command instead of being rejected or causing an overflow. An index tracks the position of the oldest entry so the buffer wraps around correctly.

4. Retrieving Entries and Displaying History

Individual entries can be retrieved by their position in the buffer. The full history is displayed in order from oldest to most recent, numbered using the total count of commands entered so far.

5. Validating Consistency

Consistency validation checks that the number of entries actually stored matches what is expected given the total number of commands added and the buffer's capacity — this confirms the wrap-around logic has not corrupted the buffer state.

6. Pipelines

A pipeline connects the output of one command to the input of the next, allowing several commands to be chained together, e.g. cmd1 | cmd2 | cmd3. Each command in the chain is a stage of the pipeline.

7. Pipeline Data Structure

A pipeline is represented as an ordered array of Command structures, where each Command holds its own argument vector. The order of the array defines the execution order of the stages.

```
typedef struct {
 char *args[MAX_ARGS];
 int argCount;
} Command;

typedef struct {
 Command stages[MAX_STAGES];
 int stageCount;
} Pipeline;
```

8. Connecting Processes with pipe() and fork()

The pipe() system call creates a unidirectional data channel with a read end and a write end. For each pair of consecutive stages, a pipe connects the write end of one process to the read end of the next, using dup2() to redirect standard input and standard output before calling execvp().

```
pipe(pipefds);
dup2(pipefds[1], STDOUT_FILENO); /* redirect this stage's output */
dup2(pipefds[0], STDIN_FILENO); /* redirect the next stage's input */
```

9. Validating Pipeline Layout

Before execution, the pipeline is checked to ensure it has at least one stage and that no stage is empty (for example, caused by malformed input such as two consecutive pipe symbols with nothing meaningful between them). An invalid layout is rejected before any process is created.

Algorithm:

Part 1 – Command History

- Start the program.
- Read a command entered by the user.
- If the history buffer is not full, append the command and increase the size.
- If the buffer is full, overwrite the oldest entry and advance the start index.
- Increase the total command counter.
- Display the history buffer in order.
- Retrieve a specific entry by its position when requested.
- Validate that the stored size matches the expected occupancy.

- Stop the program.

Part 2 – Pipeline Structures

- Start the program.
- Read the pipeline input string.
- Split the input on the pipe symbol to obtain each stage.
- Tokenize each stage into a command name and its arguments.
- Store each stage's Command structure in order inside the Pipeline structure.
- Validate that the pipeline has at least one stage and no empty stage.
- If invalid, report the error and skip execution.
- If valid, create the required pipes and fork a process per stage.
- Connect each stage's input/output to the adjacent pipes using dup2().
- Execute each stage using execvp() and wait for all stages to finish.
- Test the pipeline with single-command, multi-command and invalid configurations.
- Stop the program.

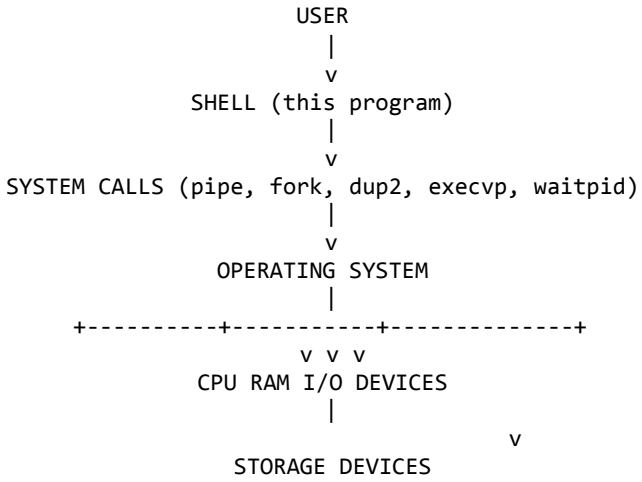
Functions Used:

Function	Purpose
`main()`	Starting point of the program
`addHistory()`	Stores a command into the circular history buffer
`displayHistory()`	Prints the stored command history in order
`getHistoryEntry()`	Retrieves a specific entry from the history buffer
`validateHistoryConsistency()`	Checks that the buffer state is internally consistent
`tokenizeCommand()`	Splits one pipeline stage into a command and its arguments
`createPipeline()`	Splits input on '|' and builds the Pipeline structure
`validatePipeline()`	Checks the pipeline has valid, non-empty stages
`printPipelineLayout()` `executePipeline()`	Displays the parsed stages of a pipeline Creates pipes, forks stages, and connects them with dup2()
`pipe()`	Creates a read/write file-descriptor pair for inter-process communication
`fork()`	Creates a child process for each pipeline stage
`dup2()`	Redirects standard input/output to a pipe end
`execvp()`	Replaces a child process image with the stage's command
`waitpid()`	Waits for each pipeline stage to complete

Hardware and Operating System Relationship

Hardware consists of physical components such as the CPU, RAM, storage devices, keyboard, mouse and display. The Operating System acts as an interface between application software and hardware. It manages processes, memory, files, devices and system resources.

Relationship



How OS Interacts with Hardware

- **Process Management** – The OS creates and schedules one process per pipeline stage. -
Memory Management – The OS manages memory for the history buffer and process images.
- **File Management** – The OS manages the file descriptors created by pipe().
- **Device Management** – The OS controls the terminal used for command input and output. -
Security – The OS enforces permissions when executing pipeline commands.

Observation

The program successfully stores each entered command into a fixed-size circular history buffer, overwrites the oldest entry once capacity is reached, displays the history in order, retrieves individual entries by position, and confirms that the buffer's occupancy stays consistent with the total number of commands entered.

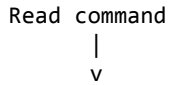
The pipeline module successfully parses multi-stage pipeline input into an ordered Pipeline structure, validates that every stage is non-empty before execution, connects consecutive stages using pipe() and dup2(), and correctly rejects a malformed pipeline containing an empty stage without attempting to execute it.

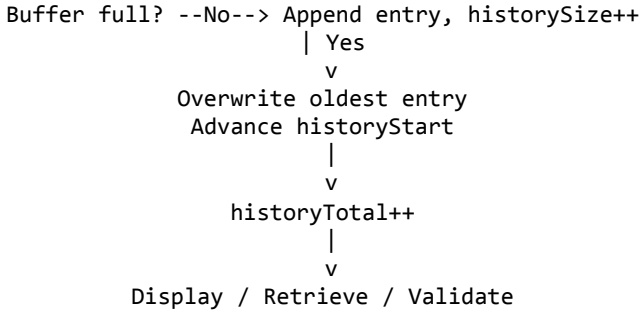
Result

Command-history management was successfully implemented, including storage, capacity handling through a circular buffer, entry retrieval, history display, and consistency validation. Pipeline structures were also successfully implemented, including pipeline creation, command storage in execution order, process connection using pipe() and fork(), layout validation, and testing across single-command, multi-command and invalid configurations.

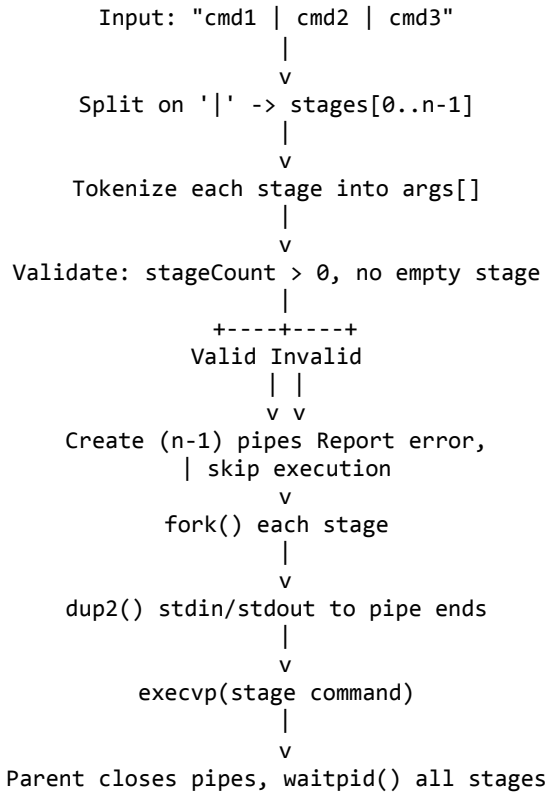
Design / Flow

Command History Flow





Pipeline Execution Flow



CODE-

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_HISTORY 5
#define MAX_CMD_LEN 200
#define MAX_STAGES 10
#define MAX_ARGS 20

/* ================= Part 1: Command History ================= */

char history[MAX_HISTORY][MAX_CMD_LEN];
int historySize = 0; /* number of entries currently stored (<= MAX_HISTORY) */ int
historyStart = 0; /* index of the oldest entry (circular buffer) */ int historyTotal
= 0; /* total number of commands ever added (for validation) */
```

```
void addHistory(const char *cmd)
{
 if (cmd == NULL || strlen(cmd) == 0)
 return;

 if (historySize < MAX_HISTORY)
 {
 strncpy(history[(historyStart + historySize) % MAX_HISTORY], cmd, MAX_CMD_LEN - 1);
historySize++;
 }
 else
 {
 /* Capacity reached: overwrite the oldest entry (circular buffer) */
strncpy(history[historyStart], cmd, MAX_CMD_LEN - 1);
 historyStart = (historyStart + 1) % MAX_HISTORY;
 }
 historyTotal++;
}

void displayHistory(void)
{
 printf("\n--- Command History (most recent last) ---\n");
 if (historySize == 0)
 {
 printf("(history is empty)\n");
 return;
 }
 for (int i = 0; i < historySize; i++)
 {
 int idx = (historyStart + i) % MAX_HISTORY;
 printf("%2d: %s\n", historyTotal - historySize + i + 1, history[idx]); }
}

/* Retrieve the nth stored entry, 1 = oldest currently stored entry */
const char *getHistoryEntry(int n)
{
 if (n < 1 || n > historySize)
 return NULL;
 int idx = (historyStart + (n - 1)) % MAX_HISTORY;
 return history[idx];
}

void validateHistoryConsistency(void)
{
 printf("\n--- Validating History Consistency ---\n");
 int expected = (historyTotal < MAX_HISTORY) ? historyTotal : MAX_HISTORY;
 if (expected == historySize)
 printf("OK: historySize (%d) matches expected buffer occupancy (%d)\n", historySize, expected); else
printf("ERROR: historySize (%d) does NOT match expected (%d)\n", historySize, expected); printf("Total
commands ever added: %d | Currently retained: %d (capacity %d)\n", historyTotal, historySize,
MAX_HISTORY);
}

void testHistory(void)
{
 printf("\n=== Part 1: Testing Command History ===\n");
 const char *cmds[] = {"pwd", "ls -l", "cd /tmp", "echo hello", "whoami", "date", "uname -a"};
int n = sizeof(cmds) / sizeof(cmds[0]);

 for (int i = 0; i < n; i++)
 {
 printf("Adding command: %s\n", cmds[i]);
 addHistory(cmds[i]);
 }

 displayHistory();
 validateHistoryConsistency();
```

```
 printf("\nRetrieving entry #2 from current buffer: %s\n", getHistoryEntry(2));
printf("Retrieving out-of-range entry #99: %s\n",
 getHistoryEntry(99) ? getHistoryEntry(99) : "(not found)");
}

/* ================= Part 2: Pipeline Structures ================= */

typedef struct {
 char *args[MAX_ARGS];
 int argCount;
} Command;

typedef struct {
 Command stages[MAX_STAGES];
 int stageCount;
} Pipeline;

void tokenizeCommand(char *segment, Command *cmd)
{
 cmd->argCount = 0;
 char *token = strtok(segment, " \t");
 while (token != NULL && cmd->argCount < MAX_ARGS - 1)
 {
 cmd->args[cmd->argCount++] = token;
 token = strtok(NULL, " \t");
 }
 cmd->args[cmd->argCount] = NULL;
}

/* Parses "cmd1 | cmd2 | cmd3" into a Pipeline structure, in execution
 order. * Uses strtok_r (reentrant) because tokenizeCommand() also calls
 strtok * internally on the " \t" delimiter -- nesting plain strtok() calls
 would * corrupt the shared internal state. */
int createPipeline(char *input, Pipeline *p)
{
 p->stageCount = 0;
 char *saveptr;
 char *segment = strtok_r(input, "|", &saveptr);
 while (segment != NULL && p->stageCount < MAX_STAGES)
 {
 tokenizeCommand(segment, &p->stages[p->stageCount]);
p->stageCount++;
 segment = strtok_r(NULL, "|", &saveptr);
 }
 return p->stageCount;
}

int validatePipeline(const Pipeline *p)
{
 if (p->stageCount == 0)
 {
 printf("Pipeline invalid: no stages found.\n");
 return 0;
 }
 for (int i = 0; i < p->stageCount; i++)
 {
 if (p->stages[i].argCount == 0)
 {
 printf("Pipeline invalid: stage %d is empty.\n", i + 1); return
0;
 }
 }
 printf("Pipeline valid: %d stage(s), %d pipe(s) needed.\n",
p->stageCount, p->stageCount - 1);
 return 1;
}

void printPipelineLayout(const Pipeline *p)
```

```
{
 for (int i = 0; i < p->stageCount; i++)
 {
 printf(" Stage %d:", i + 1);
 for (int j = 0; j < p->stages[i].argCount; j++)
 printf(" %s", p->stages[i].args[j]);
 printf("\n");
 }
}

/* Connects and executes each stage of the pipeline, linking stdout -> stdin
*/ void executePipeline(Pipeline *p)
{
 int numPipes = p->stageCount - 1;
 int pipefds[2 * MAX_STAGES];

 for (int i = 0; i < numPipes; i++)
 {
 if (pipe(pipefds + i * 2) < 0)
 {
 perror("pipe");
 return;
 }
 }

 pid_t pids[MAX_STAGES];
 for (int i = 0; i < p->stageCount; i++)
 {
 pids[i] = fork();
 if (pids[i] < 0)
 {
 perror("fork");
 return;
 }
 if (pids[i] == 0)
 {
 /* Connect input from the previous pipe (except first stage) */ if (i !=
0)
 dup2(pipefds[(i - 1) * 2], STDIN_FILENO);
 /* Connect output to the next pipe (except last stage) */
 if (i != p->stageCount - 1)
 dup2(pipefds[i * 2 + 1], STDOUT_FILENO);

 for (int j = 0; j < 2 * numPipes; j++)
 close(pipefds[j]);

 execvp(p->stages[i].args[0], p->stages[i].args);
 fprintf(stderr, "%s: command not found\n", p->stages[i].args[0]);
_exit(127);
 }
 }

 for (int j = 0; j < 2 * numPipes; j++)
 close(pipefds[j]);

 for (int i = 0; i < p->stageCount; i++)
 waitpid(pids[i], NULL, 0);
}

void testPipelineConfigurations(void)
{
 printf("\n=== Part 2: Testing Pipeline Configurations ===\n");

 char *tests[] = {
 "ls",
 "ls | wc -l",
 "ls | grep c | wc -l",
 "ls | | wc" /* deliberately malformed: empty middle stage between two pipes */ };
 int n = sizeof(tests) / sizeof(tests[0]);
```

```
 for (int i = 0; i < n; i++)
 {
 char buffer[MAX_CMD_LEN];
 strncpy(buffer, tests[i], MAX_CMD_LEN - 1);
 buffer[MAX_CMD_LEN - 1] = '\0';

 printf("\nInput: \"%s\"\n", tests[i]);
 Pipeline pipeline;
 createPipeline(buffer, &pipeline);
 printPipelineLayout(&pipeline);

 if (validatePipeline(&pipeline))
 {
 printf("Output:\n");
 executePipeline(&pipeline);
 }
 else
 {
 printf("Skipping execution due to invalid pipeline layout.\n"); }
 }
}

/* ================= Main ================= */
int main(void)
{
 setvbuf(stdout, NULL, _IONBF, 0); /* keep child (execvp) and parent output in order */
testHistory();
 testPipelineConfigurations();
 return 0;
}
```

Compilation and Execution

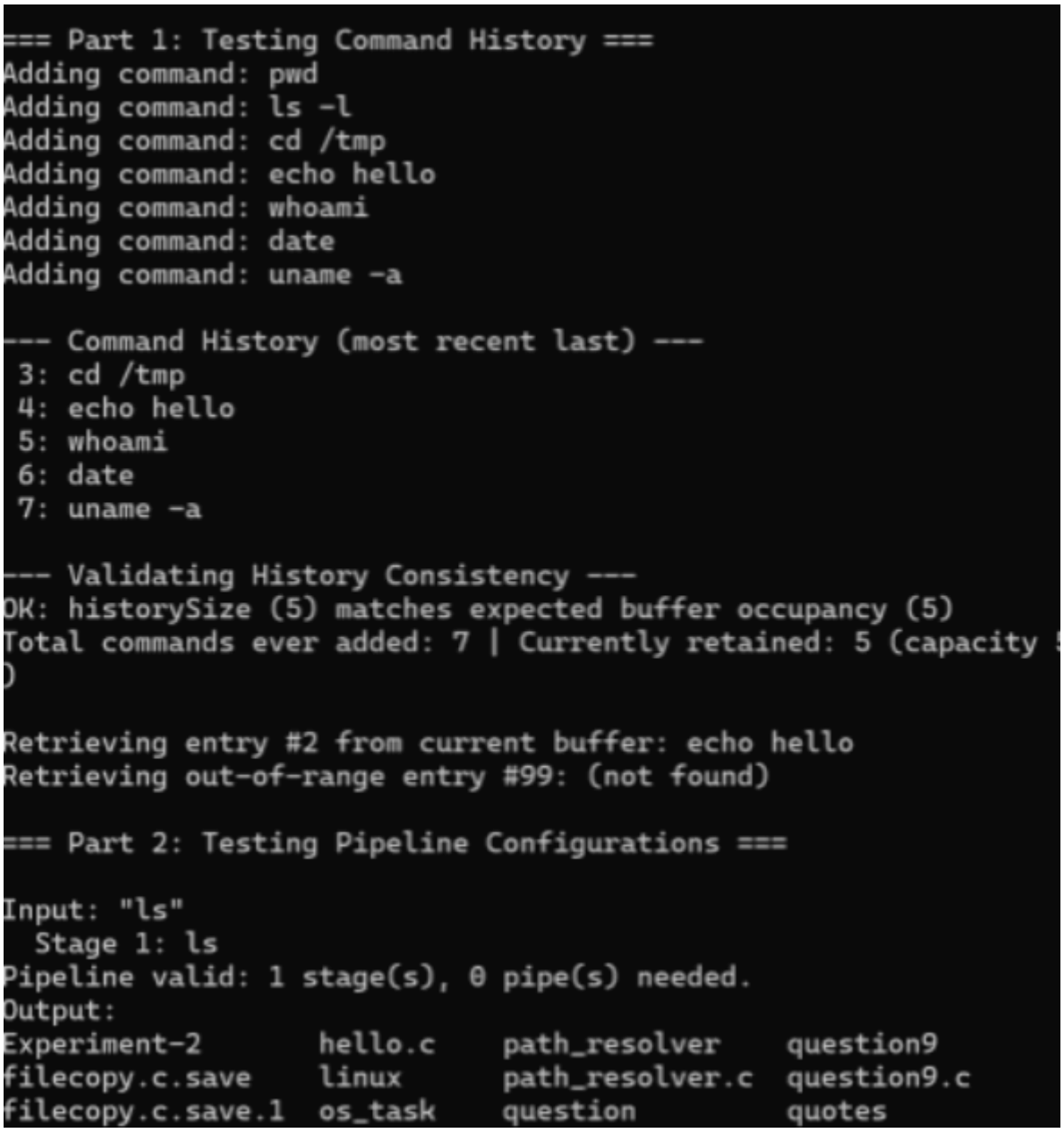
1. Compile

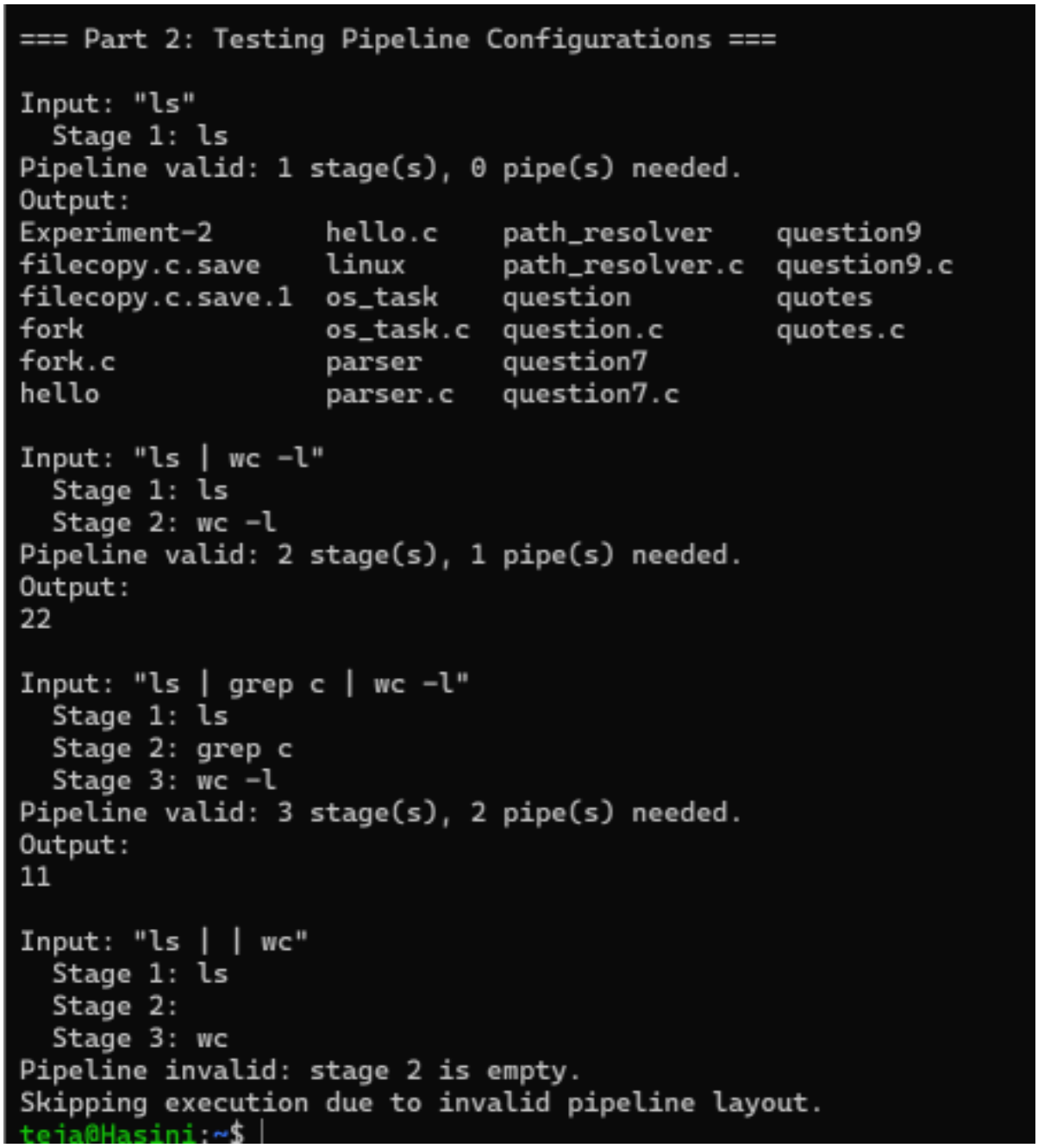
```
gcc question11.c -o question11
```

2. Execute

```
./question11
```

Sample Output


=== Part 1: Testing Command History ===
Adding command: pwd
Adding command: ls -l
Adding command: cd /tmp
Adding command: echo hello
Adding command: whoami
Adding command: date
Adding command: uname -a
--- Command History (most recent last) ---
3: cd /tmp
4: echo hello
5: whoami
6: date
7: uname -a
--- Validating History Consistency ---
OK: historySize (5) matches expected buffer occupancy (5)
Total commands ever added: 7 | Currently retained: 5 (capacity
)
Retrieving entry #2 from current buffer: echo hello
Retrieving out-of-range entry #99: (not found)
=== Part 2: Testing Pipeline Configurations ===
Input: "ls"
Stage 1: ls
Pipeline valid: 1 stage(s), 0 pipe(s) needed.
Output:
Experiment-2 hello.c path_resolver question9
filecopy.c.save linux path_resolver.c question9.c
filecopy.c.save.1 os_task question quotes
.



Conclusion

This experiment demonstrates command-history management using a circular buffer, including storage, capacity handling, entry retrieval, display and consistency validation. It also demonstrates pipeline structures that store commands in execution order and connect them into a working pipeline using pipe(), fork(), dup2() and execvp(), together with validation that rejects malformed pipeline layouts before execution. Together, these illustrate important Operating System concepts such as bounded-buffer data structures, inter-process communication, and multi-process pipeline execution.